



DESENVOLVIMENTO BACKEND

# Arquitetura de Software

Como fazer o código durar — do controlador inchado às decisões estruturais que sustentam um sistema.

# Um método, sete responsabilidades

- Lê o corpo da requisição
- Valida os dados recebidos
- Verifica pendências do estudante
- Calcula a data prevista de devolução
- Consulta o banco de dados
- Decrementa exemplares disponíveis
- Monta a resposta

**Funciona. Passa nos testes. E é exatamente esse tipo de código que, seis meses depois, ninguém quer tocar.**

A pergunta deixa de ser  
"como fazer funcionar"  
e passa a ser  
"como fazer isso durar"

É disso que trata arquitetura de software.

01

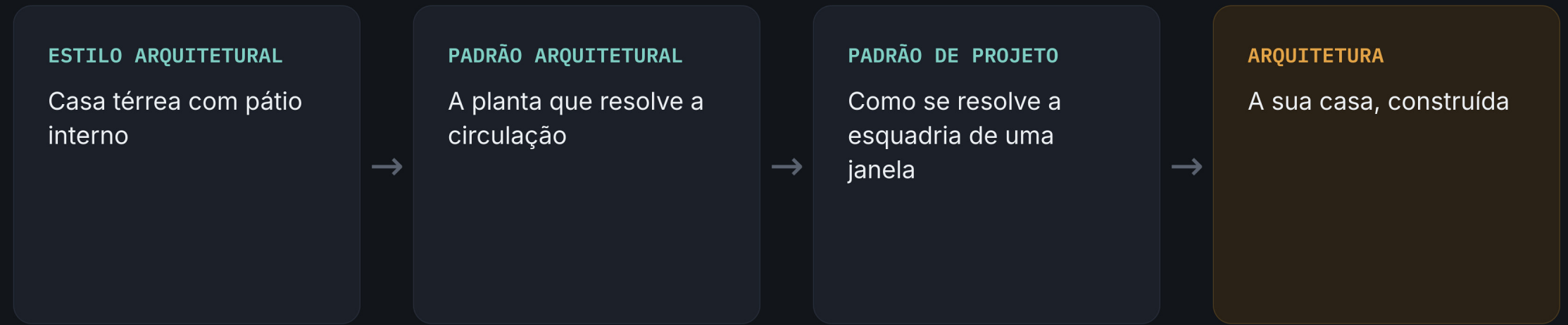
# Arquitetura, Estilo e Padrão

---

# Quatro termos que se confundem

| Termo                   | O que é                                                                                                                       |
|-------------------------|-------------------------------------------------------------------------------------------------------------------------------|
| Arquitetura de software | As decisões estruturais de um sistema específico — módulos, comunicação, tecnologias. As decisões mais difíceis de reverter.  |
| Estilo arquitetural     | Uma família de soluções, com vocabulário e restrições próprias. Cliente-servidor, camadas, microsserviços, pipes and filters. |
| Padrão arquitetural     | Uma solução recorrente para um problema recorrente de estruturação. MVC, MVVM, Hexagonal, Clean Architecture.                 |
| Padrão de projeto       | Solução recorrente em escala menor, de classes e objetos. Factory, Strategy, Observer, Repository.                            |

# Uma analogia: do abstrato ao concreto



POR QUE IMPORTA

# Vocabulário compartilhado economiza reunião

*"Vamos extrair um repositório e injetá-lo no serviço" comunica em uma frase o que levaria dez minutos para explicar.*

# O que a arquitetura serve

Um sistema pode fazer tudo o que foi pedido e ainda ser um fracasso — se for lento, se cair sob carga, se ninguém conseguir alterá-lo.

**Manutenibilidade**

**Testabilidade**

**Desempenho**

**Segurança**

**Escalabilidade**

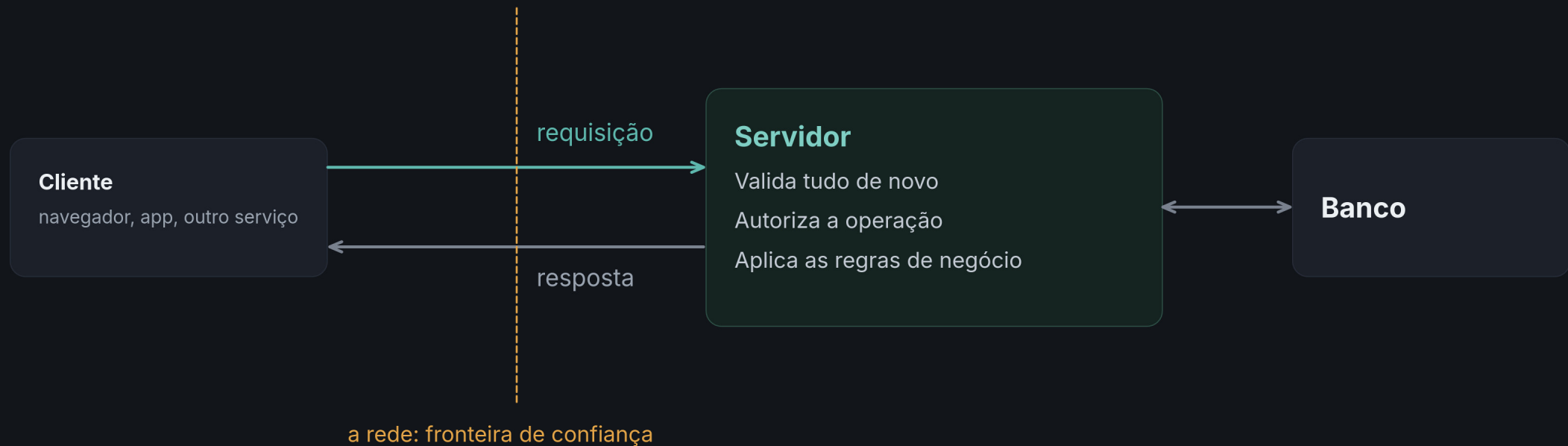
**Decididos pela arquitetura, não pelas funcionalidades.**

02

## **Estilos Arquiteturais para a Web**

---

# Cliente-servidor: o cliente pede, o servidor decide



**O servidor jamais confia no cliente.**

# Monolito

Toda a aplicação em um único artefato implantável.

## VANTAGENS

Simple de desenvolver, testar, implantar e depurar

Transações locais

Sem latência de rede interna

## LIMITES

Qualquer alteração exige reimplantar tudo

A base cresce e a compreensão fica difícil

Escala-se o conjunto, não a parte gargalo

# Microserviços

A aplicação decomposta em serviços independentes, cada um com seu banco e seu ciclo de implantação.

## VANTAGENS

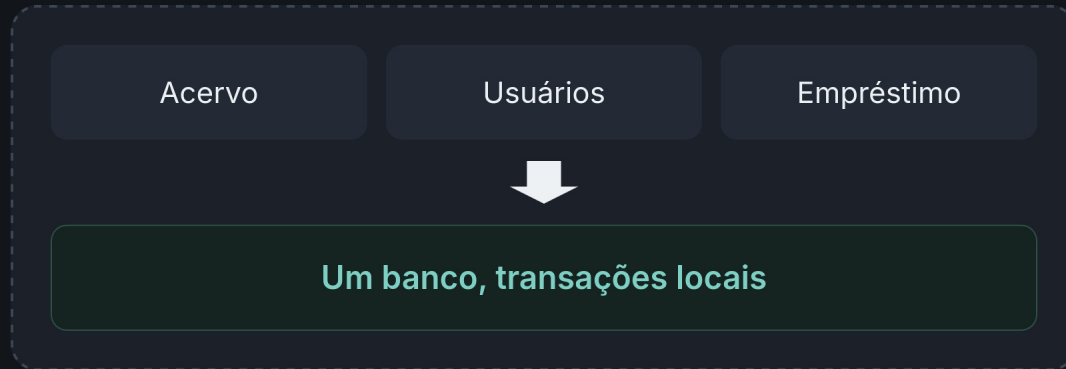
- Implantação independente
- Escala seletiva
- Times autônomos
- Isolamento de falhas

## CUSTOS

- Complexidade operacional alta
- Chamadas de rede que falham
- Consistência e depuração distribuídas
- Exige infraestrutura madura

# A mesma aplicação, duas topologias

## MONOLITO — 1 ARTEFATO



Implanta tudo junto. Escala tudo junto.

## MICROSSERVIÇOS — 3 ARTEFATOS



Cada caixa tracejada implanta e escala sozinha — e conversa pela rede, que falha.

# Microserviços resolvem problemas **organizacionais**, não técnicos.

Fazem sentido quando muitos times precisam trabalhar sem se atrapalhar. Para um sistema pequeno, mantido por poucas pessoas, são pura perda.

## RECOMENDAÇÃO DE MERCADO

Comece com um monolito bem modularizado. Decomponha só quando a dor aparecer.

# Arquitetura em camadas



# Três regras que dão sentido a tudo

01

Cada camada só conhece a de baixo

02

Nenhuma camada pula níveis

03 · teste decisivo

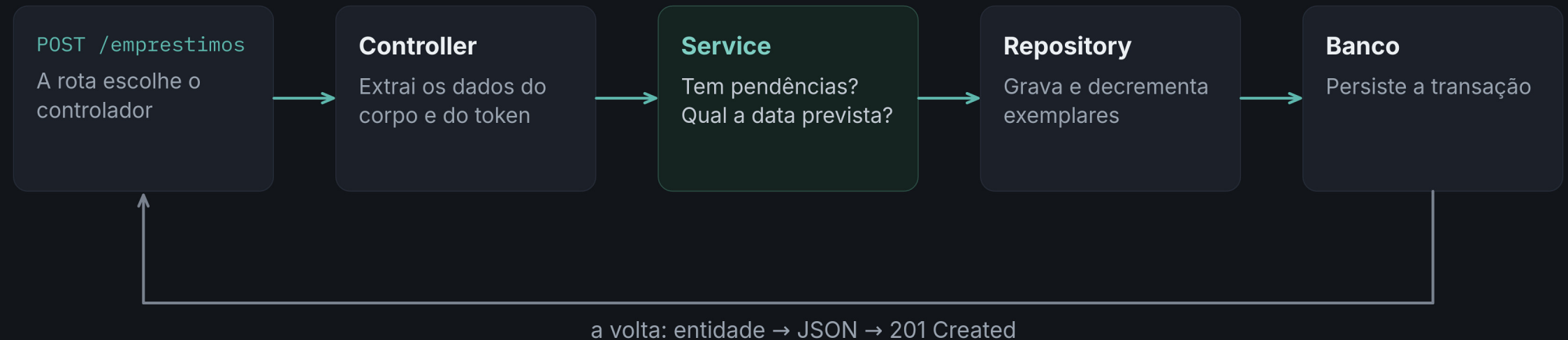
A camada de aplicação não conhece HTTP

Se a regra de negócio menciona `req.body`, ela está presa ao Express.

No dia em que precisar rodar em uma tarefa agendada ou em um evento WebSocket, será necessário reescrevê-la.

# Uma requisição atravessando as camadas

Só as duas caixas das pontas conhecem HTTP e SQL. O miolo é domínio puro.



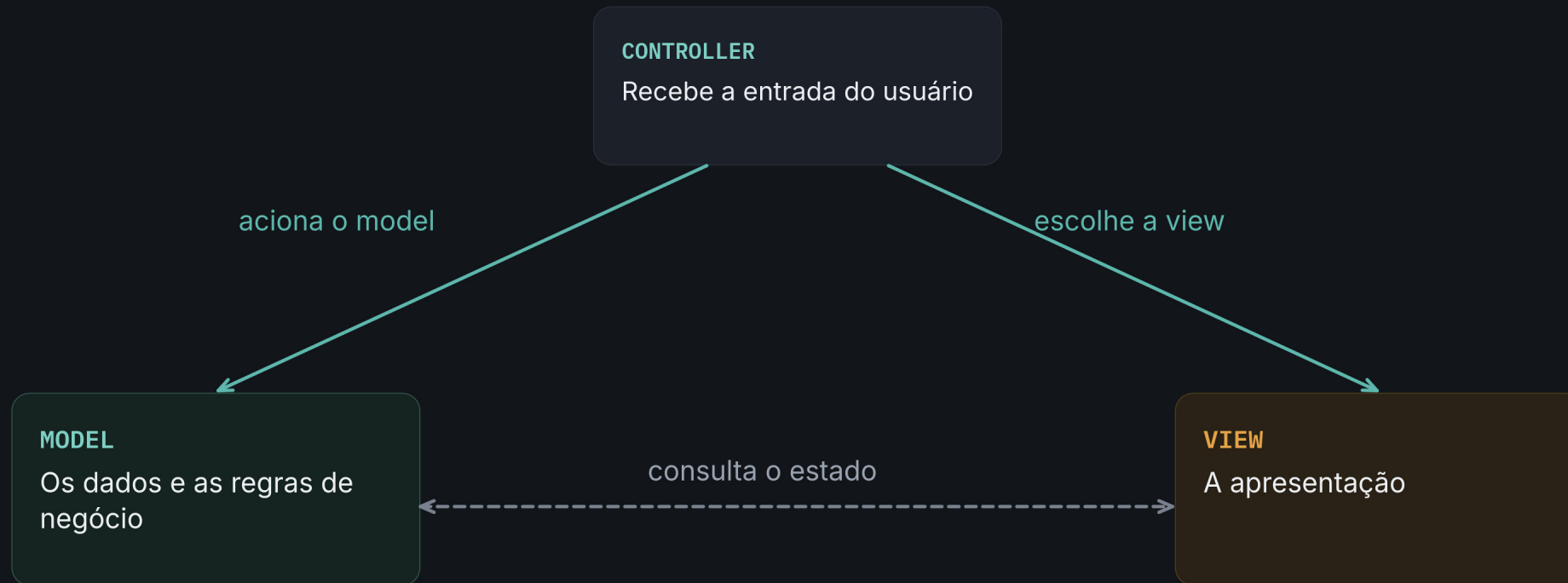
03

## **MVC e sua Adaptação a APIs**

---

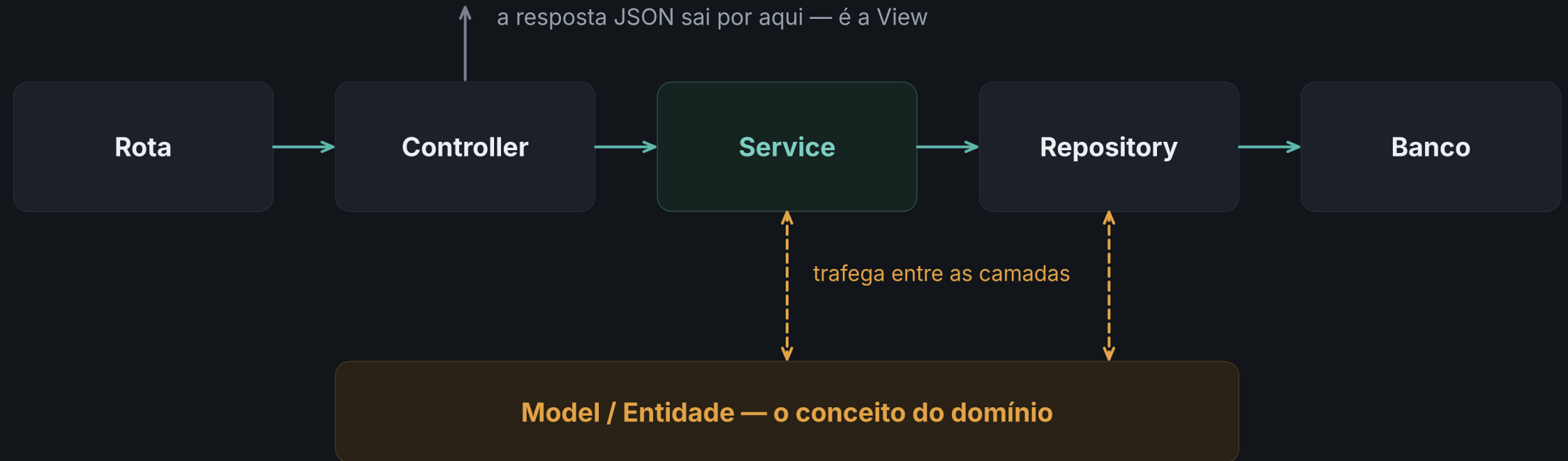
# MVC — Model, View, Controller

Formulado nos anos 1970, no Smalltalk. Um dos padrões mais influentes — e um dos mais mal compreendidos.



# Em uma API REST, onde fica a View?

A View é o JSON — ou, dependendo de como se olha, o front-end inteiro, que roda em outro lugar.



# A responsabilidade de cada peça

| Camada           | Faz                                                     | Não faz                           |
|------------------|---------------------------------------------------------|-----------------------------------|
| Rota             | Mapeia caminho e método ao controlador                  | Lógica                            |
| Controller       | Lê a requisição, chama o serviço, monta a resposta HTTP | Regra de negócio, acesso ao banco |
| Service          | Regras de negócio, orquestra repositórios, transações   | Conhecer HTTP, montar SQL         |
| Repository       | Acesso ao banco, tradução para entidades                | Regra de negócio                  |
| Model / Entidade | Representa o conceito do domínio e seu comportamento    | Conhecer banco ou HTTP            |

## O QUE LEVAR

1. Arquitetura decide o que dura, não o que funciona hoje.
2. Microsserviços resolvem problemas de time, não de código.
3. Nas camadas, a regra de negócio nunca conhece HTTP.

Perguntas?